

The Transfer Function

The central object of classical control: $G(s)=Y(s)/U(s)$.

Ogata, *Modern Control Engineering*, Ch. 2.3.

In this tutorial you will:

- define the transfer function from a linear ODE (zero initial conditions),
- build it with `tf` and inspect poles, zeros, and DC gain, and
- connect $G(s)$ to the step response.

Step through with **Ctrl+Enter**, or render a report with **publish**.

For a linear, time-invariant system described by the constant- coefficient differential equation

$$a_n \frac{d^n y}{dt^n} + a_{n-1} \frac{d^{n-1} y}{dt^{n-1}} + \dots + a_1 \dot{y} + a_0 y = b_m \frac{d^m u}{dt^m} + \dots + b_1 \dot{u} + b_0 u$$

the **transfer function** is defined as the ratio of the Laplace transform of the output to the Laplace transform of the input, with all initial conditions assumed zero:

$$G(s) = \frac{\mathbf{L}[\text{output}]}{\mathbf{L}[\text{input}]}|_{\text{zero I.C.}} = \frac{Y(s)}{U(s)} = \frac{b_m s^m + \dots + b_1 s + b_0}{a_n s^n + \dots + a_1 s + a_0}$$

The transfer function is a property of the system itself – it does not depend on the particular input applied, and it relates the output to the input only for zero initial conditions.

Contents

- Worked Example: A Second-Order Mechanical System
- Poles, Zeros, and DC Gain
- Step Response
- Try it yourself

Worked Example: A Second-Order Mechanical System

Consider a system governed by

$$2\ddot{y} + 8\dot{y} + 24y = 4u$$

Dividing by 2 and taking the Laplace transform with zero initial conditions:

$$s^2 Y(s) + 4s Y(s) + 12 Y(s) = 2U(s)$$

so

$$G(s) = \frac{Y(s)}{U(s)} = \frac{2}{s^2 + 4s + 12}$$

```
num = 2;  
den = [1 4 12];  
G = tf(num,den)
```

G =

$$\frac{2}{s^2 + 4s + 12}$$

Continuous-time transfer function.

MATLAB's `tf` object stores `num` and `den` directly. The same system can be built from its zeros, poles, and gain using `zpk`.

```
G_zpk = zpk(G)

G_zpk =

      2
-----
(s^2 + 4s + 12)

Continuous-time zero/pole/gain model.
```

Poles, Zeros, and DC Gain

The poles (roots of the denominator) govern the natural response; the zeros (roots of the numerator) shape how modes are weighted in the output.

```
z = zero(G)
p = pole(G)

z =

0×1 empty double column vector

p =

-2.0000 + 2.8284i
-2.0000 - 2.8284i
```

The DC (zero-frequency) gain is $G(0)$, found by setting $s = 0$:

```
dc_gain = dcgain(G)

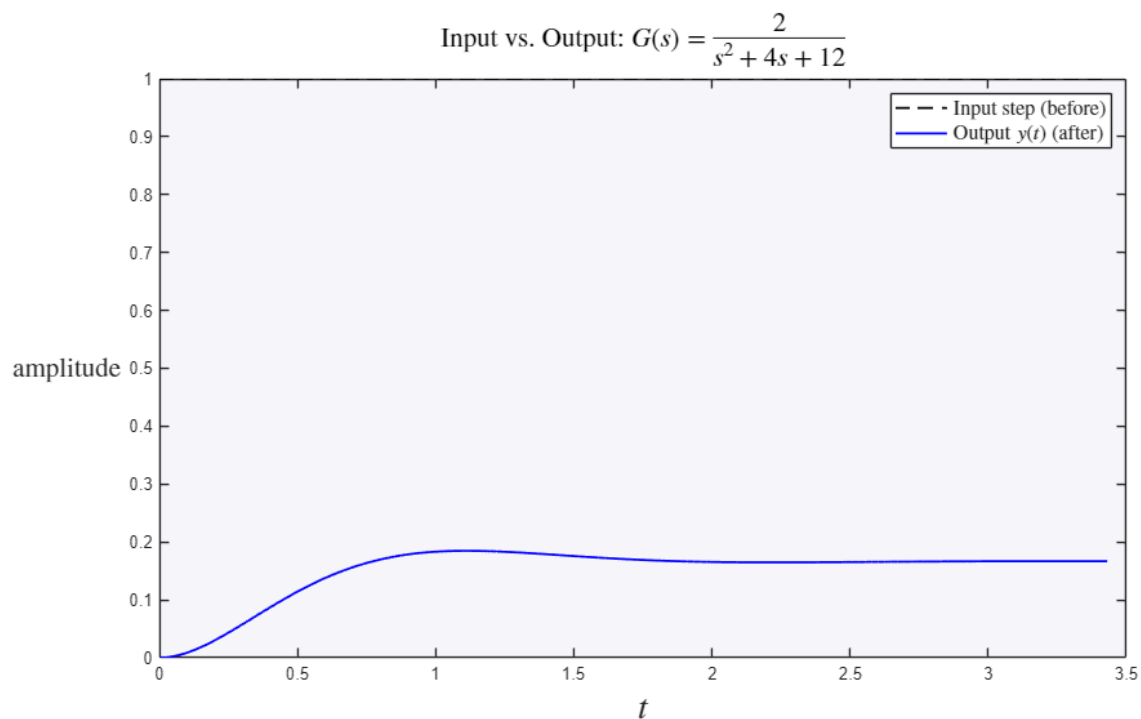
dc_gain =

0.1667
```

Step Response

For a unit step input $U(s) = 1/s$, the output transform is $Y(s) = G(s)/s$. Inverting via `step` shows the system settling to the DC gain found above.

```
figure
[ys,ts] = step(G);
plot(ts, ones(size(ts)), 'k--', ts, ys, 'b', 'LineWidth', 1.3)
legend('Input step (before)', 'Output $y(t)$ (after)', 'Interpreter', 'latex', 'FontSize', 12)
title('Input vs. Output: $G(s)=\frac{2}{s^2+4s+12}$', 'Interpreter', 'latex', 'FontSize', 16)
ylabel('amplitude', 'Interpreter', 'latex', 'FontSize', 16)
set(get(gca, 'YLabel'), 'Rotation', 0)
xlabel('$t$', 'Interpreter', 'latex', 'FontSize', 20)
```



Transfer functions of this form – built directly from a governing ODE – are the common currency used throughout the rest of this directory to model mechanical, electrical, fluid, and thermal systems.

Try it yourself

- Change the numerator to `num = [1 2]` (add a zero) and notice how the zero speeds up the early response without moving the poles.
- Compare `zpk(G)` with `tf(G)` – the same system in two readable forms.